Cell 1 (markdown)

Part A — Install Required Libraries

Cell 2 (code)

```
# Part A: Install Required Libraries

!pip install nltk scikit-learn pandas
```

Cell 3 (markdown)

Part A — Import Libraries

Cell 4 (code)

```
# Part A: Import Libraries

import nltk
import pandas as pd
import string

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

Cell 5 (markdown)

Part B — Download NLTK Resources

Cell 6 (code)

```
# Part B: Download Required NLTK Data

nltk.download('punkt')
nltk.download('stopwords')
```

Cell 7 (markdown)

Part C — Create Sample Dataset

Cell 8 (code)

```
# Part C: Expanded balanced dataset
data = {
'text': [
'I love machine learning', 'Python is great', 'I hate bugs', 'Amazing movie', 'Terrible food',
'I enjoy NLP', 'Bad service', 'AI is fascinating', 'Happy results', 'Horrible experience',
'Fantastic day', 'I dislike errors', 'Great performance', 'Very disappointing', 'Superb quality',
'The code is clean', 'I am bored', 'This is the best', 'I am angry', 'Wonderful work',
```

```
'Not good at all', 'I feel successful', 'Waste of time', 'Brilliant idea', 'I am sad',
'It is failing', 'Worst product ever', 'I am delighted', 'So painful', 'Pure joy',
'Awful result', 'Really bad', 'Total disaster', 'Extremely poor', 'Not happy'
],
'label': [1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0]
}

df = pd.DataFrame(data)
print(f'Dataset size: {len(df)}')
display(df.tail())
```

Cell 9 (markdown)

```
## Part D — Text Preprocessing
```

Cell 10 (code)

```
import nltk
# Part D: Text Preprocessing

import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download('punkt_tab', quiet=True)

stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
text = str(text).lower()
tokens = word_tokenize(text)
tokens = [word for word in tokens if word.isalpha()]
tokens = [word for word in tokens if word not in stop_words]
tokens = [stemmer.stem(word) for word in tokens]
return " ".join(tokens)

# Applying to the full 35-row dataframe
df['processed_text'] = df['text'].apply(preprocess_text)

print(f'Processed {len(df)} rows.')
print(df[['text', 'processed_text']].tail())
```

Cell 11 (markdown)

```
## Part E — TF-IDF Vectorization
```

Cell 12 (code)

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

Cell 13 (code)

```
from sklearn.feature_extraction.text import CountVectorizer
# Part E: Count Vectorization

vectorizer = CountVectorizer()

# Fit on all 35 processed rows
X = vectorizer.fit_transform(df['processed_text'])
y = df['label']

print("Current Vector Shape:", X.shape)
```

Cell 14 (markdown)

```
## Part F — Train-Test Split
```

Cell 15 (code)

```
# Part F: Train-Test Split

# Split based on all 35 samples
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)

print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples: {X_test.shape[0]}")
```

Cell 16 (markdown)

```
## Part G — Train Naive Bayes Classifier
```

Cell 17 (code)

```
from sklearn.ensemble import RandomForestClassifier
# Part G: Re-training Random Forest on the synchronized 35-sample split

model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
model.fit(X_train, y_train)

print("Random Forest Model successfully retrained on synchronized data.")
```

Cell 18 (markdown)

```
## Part H — Prediction
```

Cell 19 (code)

```
# Part H: Prediction

# Refresh predictions using the updated Random Forest
y_pred = model.predict(X_test)
```

```
print(f"Predicted labels: {y_pred}")
print(f"Actual labels: {y_test.values}")
```

Cell 20 (markdown)

```
## Part I — Evaluation
```

Cell 21 (code)

```
# Part I: Evaluation
from sklearn.metrics import accuracy_score, classification_report

# Refreshing metrics to reflect the current test set (7 samples)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, zero_division=0)

print(f"Final Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(report)
```

Cell 22 (markdown)

```
## Part J — Test New Sentence
```

Cell 23 (code)

```
# Part J: Test New Sentence

new_text = ["Python is greate"]

# Preprocess
processed = [preprocess_text(text) for text in new_text]

# Convert into TF-IDF
vector = vectorizer.transform(processed)

# Predict
prediction = model.predict(vector)

print("Input Sentence:", new_text[0])

if prediction[0] == 1:
print("Predicted Sentiment: Positive")
else:
print("Predicted Sentiment: Negative")
```

Cell 24 (code)

```
print(f"Processed input: {processed}")
# Check if words from the input are in the vectorizer vocabulary
for word in processed[0].split():
status = "in" if word in vectorizer.vocabulary_ else "NOT in"
print(f"Word '{word}' is {status} the vocabulary.")
```

```
if 'good' in stop_words:
print("Note: 'good' is in the stop_words list and was removed.")
```